

Matheus Malaman

Formativa 08

Limeira – SP

2026

Sumário

5. Documentação	3
5.1. Diagrama de Classes	3
5.2. Documentação de API	4
5.2.1. Swagger	4
5.2.2. Postman	5
5.2.3. Insomnia	6
Síntese de Aplicação Integrada (Workflow do Projeto)	6

Estrutura de Documentação do Sistema

Objetivo deste Documento: Apresentar a fundamentação teórica sobre os artefatos de documentação de software selecionados e definir as diretrizes práticas de sua usabilidade e integração no escopo do nosso projeto.

5. Documentação

A documentação é um pilar fundamental no ciclo de vida do desenvolvimento de software (SDLC). Ela atua como a única fonte de verdade para arquitetos, desenvolvedores, testadores e stakeholders, garantindo que o sistema seja compreendido, mantido e escalado com eficiência e baixo risco de falhas de comunicação.

5.1. Diagrama de Classes

O Diagrama de Classes é o diagrama central da UML (Unified Modeling Language). Ele descreve a estrutura estática do sistema, apresentando as classes, seus atributos, métodos e os relacionamentos entre elas (como herança, associação, agregação e composição).

- **Conceito-Chave:** É a representação visual do modelo de domínio e da arquitetura orientada a objetos. Ele traduz os requisitos de negócios em componentes lógicos de software estruturados.
- **Benefícios:**
 - Facilita a compreensão da arquitetura do sistema antes do início da codificação.
 - Serve como base para o design do banco de dados (mapeamento objeto-relacional).
 - Identifica gargalos de acoplamento e dependências complexas precocemente.

Diretrizes de Usabilidade em Nosso Projeto: No nosso projeto, o Diagrama de Classes será utilizado na fase de design e planejamento (antes da implementação de novas *features*). Ele servirá para alinhar a visão da equipe de engenharia sobre como as entidades do sistema interagem. Toda nova entidade ou refatoração estrutural profunda deverá ser precedida por uma atualização neste diagrama, garantindo que a equipe de banco de dados e a de *backend* estejam em total sincronia.

5.2. Documentação de API

A Documentação de API é o manual de instruções técnico que descreve como utilizar e integrar-se a uma Interface de Programação de Aplicações (API). Ela detalha os endpoints disponíveis, os métodos HTTP permitidos (GET, POST, PUT, DELETE), os formatos de requisição e resposta (geralmente JSON), os códigos de status HTTP e os requisitos de autenticação.

- **Conceito-Chave:** A documentação da API atua como um **contrato** formal entre o *backend* (provedor dos dados) e o *frontend* ou serviços de terceiros (consumidores dos dados).

Diretrizes de Usabilidade em Nosso Projeto: Para o nosso projeto, a documentação da API será tratada como um artefato vivo. O desenvolvimento será orientado a contratos (API-First), o que significa que as equipes de *frontend* e *backend* concordarão com o formato dos dados (através da documentação) antes do desenvolvimento da lógica, permitindo trabalho paralelo e reduzindo atritos na integração.

Para gerenciar, projetar e testar essas APIs, utilizaremos as seguintes ferramentas:

5.2.1. Swagger

O Swagger (baseado na especificação OpenAPI) é um ecossistema de ferramentas que permite projetar, construir, documentar e consumir serviços web RESTful. Ele gera uma interface visual interativa (Swagger UI) diretamente a partir do código-fonte ou de um arquivo de configuração (YAML/JSON).

- **Características Principais:**
 - * **Interatividade:** Permite que os desenvolvedores enviem requisições reais para a API diretamente pelo navegador.
 - **Automação:** Mantém a documentação sempre atualizada com o código, reduzindo o esforço manual.
- **Usabilidade no Nosso Projeto:** O Swagger será a nossa **vitruve oficial da API**. Ele será acoplado ao nosso código *backend* para gerar a documentação automaticamente. Qualquer novo *endpoint* criado deverá conter as anotações do Swagger para que a interface reflita imediatamente as mudanças. Servirá como consulta primária para a equipe de *frontend* e para futuros parceiros externos.

5.2.2. Postman

O Postman é uma plataforma líder no mercado para construção, teste e uso de APIs. Embora tenha começado como um simples cliente REST, evoluiu para uma ferramenta completa de colaboração.

- **Características Principais:**
 - Criação de *Collections* (coleções de requisições organizadas por módulos).
 - Gestão de variáveis de ambiente (desenvolvimento, homologação, produção).
 - Criação de testes automatizados e *mocks* (simulações de respostas).
- **Usabilidade no Nosso Projeto:** O Postman será utilizado para testes exploratórios, automação de testes de integração e compartilhamento de fluxos complexos. A equipe de QA (Garantia de Qualidade) e os desenvolvedores criarão uma *Collection* oficial do projeto. Essa *Collection* será exportada e compartilhada (ou mantida em um *workspace* da equipe) para facilitar o *onboarding* de novos desenvolvedores, permitindo que eles testem cenários completos (ex: Login -> Gerar Token -> Cadastrar Item) em poucos cliques.

5.2.3. *Insomnia*

O Insomnia é um cliente de API (REST, GraphQL, gRPC) focado no design e consumo, conhecido por sua interface minimalista, leveza e alta performance.

- **Características Principais:**
 - Excelente suporte nativo para GraphQL e desenvolvimento *design-first*.
 - Interface limpa e livre de distrações, com excelente gerenciamento de ambientes estritos.
 - Forte apelo à privacidade e operação offline.
 - **Usabilidade no Nosso Projeto:** O Insomnia atuará como a **ferramenta de produtividade individual** e uma alternativa oficial ao Postman para desenvolvedores que preferem um cliente mais ágil, leve e focado estritamente no consumo e *debug* de *endpoints* durante a codificação diária. Ele será especialmente recomendado durante a etapa de modelagem inicial da API (Design) e na prototipagem de consultas complexas.
-

Síntese de Aplicação Integrada (Workflow do Projeto)

Para garantir o sucesso do projeto, o fluxo de trabalho com essas tecnologias se dará da seguinte forma:

1. **Planejamento:** O arquiteto desenha o **Diagrama de Classes** (5.1) para estruturar o banco e as entidades.
2. **Desenvolvimento:** O *backend* implementa o código e utiliza as anotações do **Swagger** (5.2.1) para gerar a documentação interativa automaticamente.
3. **Testes e Consumo Individual:** Os desenvolvedores utilizam o **Insomnia** (5.2.3) localmente por sua rapidez para debugar e validar as rotas criadas.
4. **Colaboração e QA:** Fluxos completos de requisições e testes automatizados de API são consolidados e compartilhados com toda a equipe através de Collections no **Postman** (5.2.2).

